ROLE SUMMARY:

Responsible for the entire **data layer of the Restaurant Management System (RMS)**.

Designed and implemented the MongoDB database structure, including schemas for restaurant operations, staff management, reservations, menu, orders, inventory, billing, payments, attendance, leaves, salary records, reviews, complaints, and notifications.

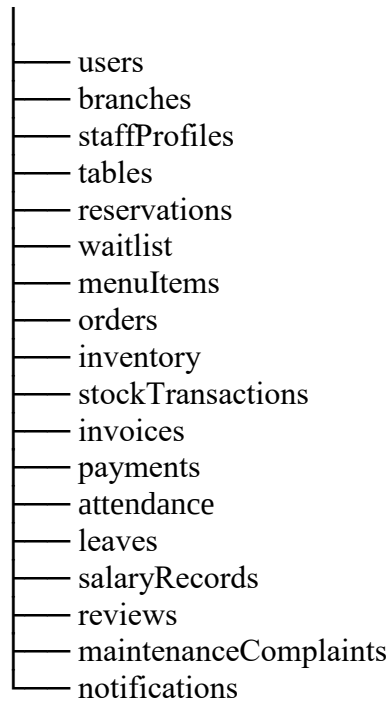
Responsibility includes:

- Designing MongoDB/Mongoose schemas
- Establishing the MongoDB database connection
- Managing relationships between collections using ObjectId references
- Configuring Cloudinary for menu/image storage
- Creating database seed/sample data
- Maintaining database configuration and environment variables
- Ensuring the backend has a proper and scalable data structure

The main database is:

Database Name: restaurantDB

The database contains the following collections:

restaurantDB


SECTION 1: DATABASE MODELS

server/models/

These Mongoose schema files define the structure of the collections used by the Restaurant Management System.

1. server/models/User.js

WHAT IT DOES:

Defines the main user/account schema used for authentication and system access.

Contains information such as:

- name
- email
- password
- role
- account status
- authentication-related information
- timestamps

Roles can include restaurant system users such as:

- admin
- manager
- waiter
- cashier
- kitchen

WHY IT MATTERS:

The User collection is the foundation of authentication and role-based access control.

Login, authorization, protected routes, and staff access depend on this collection.

2. server/models/Branch.js

WHAT IT DOES:

Defines the restaurant branch information.

Contains information such as:

- branch name
- address
- contact information
- opening/closing information
- branch status
- timestamps

WHY IT MATTERS:

The Branch collection allows the system to support **multiple restaurant branches**.

Other collections can associate their records with a particular branch.

3. server/models/StaffProfile.js

WHAT IT DOES:

Stores detailed information about restaurant employees separately from their login account.

Contains information such as:

- user reference
- branch reference
- employee information
- designation/position
- contact information
- joining information
- employment status

WHY IT MATTERS:

The users collection handles the account/authentication side, while staffProfiles handles detailed employee information.

This separation makes the staff management system easier to maintain.

4. server/models/Table.js

WHAT IT DOES:

Defines the restaurant dining table structure.

Contains information such as:

- table number
- branch reference
- seating capacity
- table status
- location/zone
- current order reference

Possible table statuses include:

- available
- occupied
- reserved
- cleaning

WHY IT MATTERS:

Tables are required for dine-in operations, reservations, order management, and restaurant floor management.

5. server/models/Reservation.js

WHAT IT DOES:

Stores customer table reservation information.

Contains information such as:

- customer name
- phone/email
- branch
- reservation date
- time slot
- party size
- table reference
- reservation status
- special notes

WHY IT MATTERS:

The Reservation collection manages advance bookings and connects customers with restaurant tables.

6. server/models/Waitlist.js

WHAT IT DOES:

Stores customers waiting for a table when no suitable table is immediately available.

Contains information such as:

- customer name
- contact information
- party size
- branch
- waiting status
- estimated waiting information
- created time

WHY IT MATTERS:

The waitlist helps restaurant staff manage customers during busy periods when all tables are occupied.

7. server/models/MenuItem.js

WHAT IT DOES:

Defines the restaurant menu item structure.

Contains information such as:

- item name
- description
- price
- category
- image
- availability
- dietary information
- branch information where applicable
- timestamps

Images can be stored using **Cloudinary URLs**.

WHY IT MATTERS:

Menu items are directly used by:

- customer menu
- order creation
- billing
- kitchen operations
- reports
- reviews
- restaurant analytics

8. server/models/Order.js

WHAT IT DOES:

Defines the main restaurant order structure.

An order can contain multiple menu items.

Contains information such as:

- table reference
- customer information where applicable
- branch reference
- ordered items
- quantity
- item price
- item notes
- order status
- staff/user references
- subtotal

- tax
- total amount
- payment status
- timestamps

WHY IT MATTERS:

The Order collection is one of the **core business collections** of the RMS.

It connects:

Tables → Menu Items → Kitchen → Billing → Payments → Reports

9. server/models/Inventory.js

WHAT IT DOES:

Stores information about restaurant ingredients and stock.

Contains information such as:

- item name
- quantity
- unit
- minimum stock threshold
- branch
- current stock status
- last restocked information

WHY IT MATTERS:

Inventory management helps the restaurant monitor available ingredients and identify low-stock items.

10. server/models/StockTransaction.js

WHAT IT DOES:

Records every major stock movement.

Examples include:

- stock added
- stock removed
- stock adjustment
- purchase/restocking
- consumption

Contains information such as:

- inventory item reference
- transaction type
- quantity
- previous stock
- updated stock
- user/staff reference
- branch
- transaction date

WHY IT MATTERS:

The Inventory collection shows the current stock, while StockTransactions maintain the **history of how that stock changed**.

11. server/models/Invoice.js

WHAT IT DOES:

Stores billing/invoice information generated from restaurant orders.

Contains information such as:

- order reference
- invoice number
- customer information
- item details
- subtotal
- tax
- discount where applicable
- total amount
- invoice status
- generated date

WHY IT MATTERS:

Invoices provide a formal record of restaurant billing and connect orders with financial records.

12. server/models/Payment.js

WHAT IT DOES:

Stores payment transactions.

Contains information such as:

- invoice/order reference
- payment amount

- payment method
- payment status
- transaction/reference information
- payment date
- received by user/staff

Payment methods can include:

- Cash
- Card
- UPI

WHY IT MATTERS:

The Payment collection maintains the financial transaction history of the restaurant.

SECTION 2: STAFF & HR DATABASE MODELS

13. server/models/Attendance.js

WHAT IT DOES:

Stores employee attendance records.

Contains information such as:

- staff reference
- branch
- date
- check-in time
- check-out time
- attendance status

WHY IT MATTERS:

Attendance records are required for staff management and salary calculation.

14. server/models/Leave.js

WHAT IT DOES:

Stores employee leave requests and leave history.

Contains information such as:

- staff reference
- leave type
- start date

- end date
- reason
- approval status
- approved/rejected by
- timestamps

WHY IT MATTERS:

The Leave collection allows managers/admins to manage employee leave requests.

15. server/models/SalaryRecord.js

WHAT IT DOES:

Stores employee salary information and salary records.

Contains information such as:

- staff reference
- salary period
- basic salary
- deductions
- bonuses/additions
- net salary
- payment status
- payment date

WHY IT MATTERS:

Salary records connect employee management with payroll operations.

Attendance and leave information can also be used when calculating salary.

SECTION 3: CUSTOMER & SUPPORT MODELS

16. server/models/Review.js

WHAT IT DOES:

Stores customer reviews and feedback.

Contains information such as:

- customer name/reference
- rating
- comment
- menu item reference where applicable
- order reference

- branch
- created date

WHY IT MATTERS:

Reviews help the restaurant collect customer feedback and evaluate menu items and service.

17. server/models/MaintenanceComplaint.js

WHAT IT DOES:

Stores maintenance-related complaints reported inside the restaurant.

Examples:

- AC problem
- lighting problem
- table/chair damage
- plumbing issue
- electrical issue
- equipment problem

Contains information such as:

- complaint title
- description
- branch
- reported by
- assigned staff
- priority
- status
- created/updated timestamps

WHY IT MATTERS:

This collection helps management track restaurant maintenance issues from reporting to resolution.

18. server/models/Notification.js

WHAT IT DOES:

Stores system notifications for users/staff.

Notifications can be generated for events such as:

- new order
- reservation
- low inventory
- leave request
- maintenance complaint
- payment/billing update
- other important system events

Contains information such as:

- recipient/user reference
- notification type
- title
- message
- read/unread status
- related record reference
- timestamps

WHY IT MATTERS:

Notifications keep staff and management updated about important restaurant activities.

SECTION 4: DATABASE CONNECTION & CONFIGURATION

server/config/

server/config/db.js

WHAT IT DOES:

Handles the MongoDB connection using Mongoose.

The connection reads the MongoDB connection string from the .env file.

Example:

MONGO_URI=your_mongodb_connection_string

The configuration handles:

- MongoDB connection
- connection errors
- successful connection logging
- database initialization

The application connects to:

restaurantDB

WHY IT MATTERS:

Without the MongoDB connection, the backend cannot read or write restaurant data.

server/config/cloudinary.js

WHAT IT DOES:

Configures Cloudinary for image storage.

Credentials are loaded from environment variables instead of being written directly into the source code.

Example variables:

CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=

WHY IT MATTERS:

Cloudinary is used for storing restaurant/menu images while MongoDB stores the corresponding image URL.

This prevents large image files from being stored directly inside MongoDB.

SECTION 5: ENVIRONMENT CONFIGURATION

server/.env.example

WHAT IT DOES:

Provides a safe template of the environment variables required by the backend.

It should contain placeholders such as:

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_jwt_secret

CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

IMPORTANT:

Never upload the actual .env file containing real passwords, API keys, database credentials, or secrets to GitHub.

Only .env.example should be committed.

SECTION 6: DATA SEEDING SCRIPTS

server/seeds/

server/seeds/seed.js

WHAT IT DOES:

The main database initialization script.

It can be used to create initial/sample records for the Restaurant Management System, including:

- users
- branches
- staff profiles
- restaurant tables
- menu items
- inventory
- reservations
- sample operational data where required

WHY IT MATTERS:

The seed script allows the development team to quickly create a usable database instead of manually entering every record through MongoDB Compass.

server/seeds/seedOrders.js

WHAT IT DOES:

Creates sample order-related data for testing.

It can generate sample:

- orders
- invoices
- payments
- order history

WHY IT MATTERS:

Sample order data is useful for testing:

- Dashboard
- Revenue reports
- Billing
- Payment history

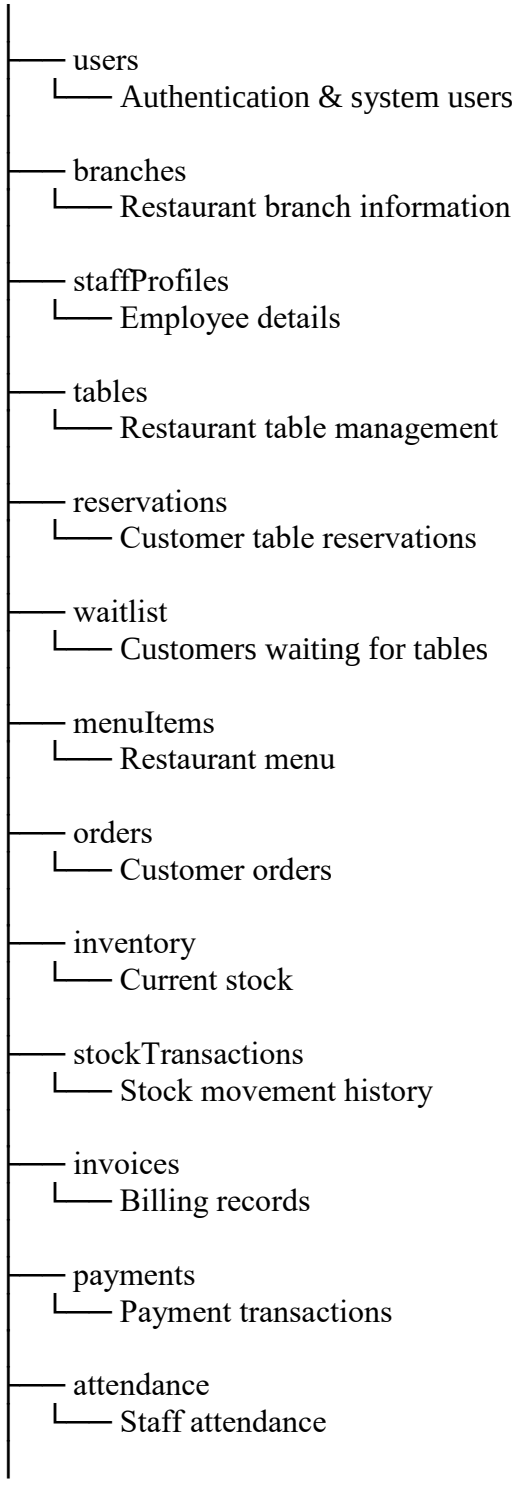
- Analytics
- Order management

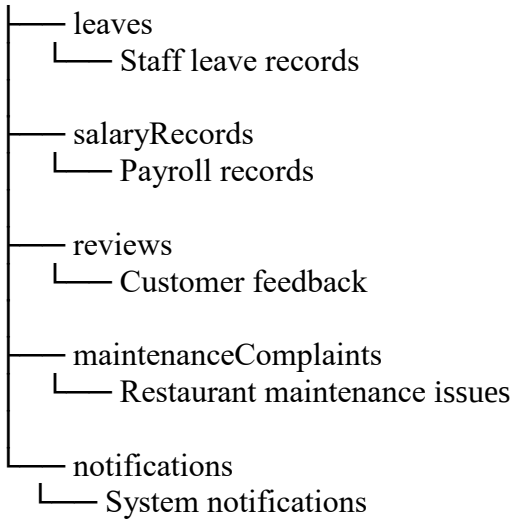
Without sample data, dashboard statistics may show empty or zero values.

SECTION 7: MONGODB DATABASE STRUCTURE

The final MongoDB database structure is:

restaurantDB

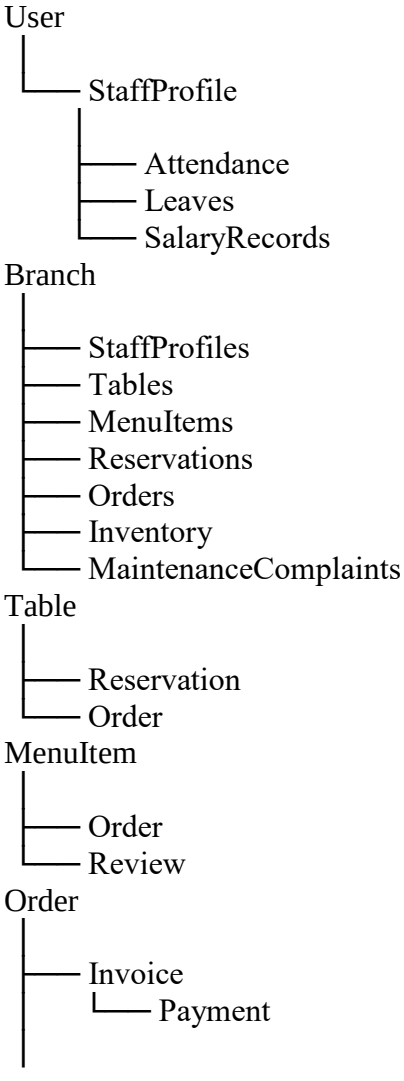




SECTION 8: IMPORTANT COLLECTION RELATIONSHIPS

The database is designed using MongoDB references between related documents.

Important relationships include:



└── Review
Inventory
│
└── StockTransactions
User
│
└── Notifications

These relationships allow different RMS modules to share consistent data without unnecessarily duplicating records.

HOW TO COMMIT ON GITHUB

Commit 1 — User & Branch Management

Files:

server/models/User.js
server/models/Branch.js
server/models/StaffProfile.js

Commit message:

feat: designed User, Branch, and StaffProfile MongoDB schemas

Commit 2 — Table & Reservation Management

Files:

server/models/Table.js
server/models/Reservation.js
server/models/Waitlist.js

Commit message:

feat: added Table, Reservation, and Waitlist data models

Commit 3 — Menu & Order Management

Files:

server/models/MenuItem.js
server/models/Order.js

Commit message:

feat: architected MenuItem and Order data models

Commit 4 — Inventory Management

Files:

server/models/Inventory.js
server/models/StockTransaction.js

Commit message:

feat: implemented Inventory and StockTransaction schemas

Commit 5 — Billing & Payment

Files:

server/models/Invoice.js
server/models/Payment.js

Commit message:

feat: added Invoice and Payment transaction schemas

Commit 6 — Staff HR & Payroll

Files:

server/models/Attendance.js
server/models/Leave.js
server/models/SalaryRecord.js

Commit message:

feat: implemented Attendance, Leave, and SalaryRecord schemas

Commit 7 — Reviews, Complaints & Notifications

Files:

server/models/Review.js
server/models/MaintenanceComplaint.js
server/models/Notification.js

Commit message:

feat: added Review, MaintenanceComplaint, and Notification schemas

Commit 8 — Database & Cloudinary Configuration

Files:

server/config/db.js
server/config/cloudinary.js
server/.env.example

Commit message:

chore: configured MongoDB connection and Cloudinary storage

Commit 9 — Database Seed Data

Files:

server/seeds/seed.js
server/seeds/seedOrders.js

Commit message:

chore: added database seed scripts for RMS sample data